1. Introduction

Automated “bot” accounts on social media platforms pose a significant threat to society by spreading disinformation and manipulating online discourse. This project addresses the critical task of discerning bots from humans, framed as an anomaly detection problem where bots represent the minority class. My approach to this competition progresses from exploratory data analysis (EDA), data processing and feature engineering, developing and validating models, additional techniques to help improve the model, and model evaluations to determine which final model to choose. For more details on the process, refer to the notebook E1121498.ipynb.

2. Methodology

2.1. Exploratory Data Analysis

Initial EDA revealed that the only columns with missing values include description, lang, location, and both url features. These missing values need to be handled later on, which seem to mainly be textual features. The target distribution was also plotted, to show that about 33% of the train data are bots, which indicate a skewed dataset.

A feature by feature analysis yielded several key insights. The ID feature was found to have been generated with a combination of data center code, datetime, and a sequential number. Since location captures positional information and created_at or account age days captures temporal information, and should therefore drop it.

A correlation matrix of numerical features using Pearson’s correlation coefficient revealed that most features have relatively low correlation, except for average_tweets_per_day and statuses_count having 0.94 correlation. However, we already know that more robust models will need to be used for this competition due to the skewed dataset, and those models are likely to be robust to high correlation. Numeric feature distributions were also heavily skewed, and transformations may be helpful for the model. A Q-Q plot against normal distribution, revealed that log transforms significantly help the distributions.

Analysis of binary features show that geo_enabled and verified are likely strong negative predictors of target. While for default_profile_image = True, there are more bots than non-bots, but very few accounts have default_profile_image = True. We also plot a correlation matrix as done with the numerical features, and find that there is not much correlation between binary features.

Among other features, created_at is used to plot and show the distribution of account creation dates, but carries the same information as account_age_days, so it should be dropped. location is the only categorical variable, I considered it as such since there were

only about 48 unique values, which can be one-hot encoded since there are not too many unique values. After encoding there would be 49 additional values taking note of the empty values mentioned above. For the textual features like lang, description, screen_name, and the url features, these features will be much harder to work on, and might have a lot of noise if using it to train models. Some models will be trained without the textual data and some with later on to evaluate its usefulness.

2.2. Data Preprocessing and Feature Engineering

For this section, we will feature engineer first, since features will be dropped in data preprocessing and we want to use those to engineer other features first. Feature engineering was split into text feature engineering, interaction feature engineering, transformations and natural language processing (NLP). More in depth explanations for each feature can be found in the notebook as well.

Text feature engineering included the length of each text feature, which would likely help in bot detection for screen_name and description. Next was textual pattern features, such as whether a url exists within the description, whether the screen name has a number or whether location or description is empty.

Interaction feature engineering included activity normalized features, social ratios, binary and continuous hybrid features, binary against binary features, length ratios and a composite score. Activity normalized features normalizes metrics by account lifetime to detect abnormal activity rates, like followers, friends, statuses and favourites per day. Social ratios include interaction patterns between features, like followers to friends or favourites to statuses which give us information on how accounts interact with others which could be a strong bot indicator. Binary and continuous hybrids attempt to capture conditional relationships, like verified * followers, which is a strong human indicator if it is a non-zero and large integer. Binary against binary include geo enabled * verified, since both of them are strong human account predictors, combining their signal might improve its signal. Length ratios like description to screen name length might provide information about the relative effort put into filling out the profile. Lastly, the composite score gives one point for each factor of an account that has a description, has a location, has a custom profile and profile image, and is geo enabled, with the maximum score adding up to five.

For transformations, the numeric count features were log transformed and added as additional features, to allow for the models to choose which features to drop when developing them later, for example, only the log transformed count features were kept for the logistic regression, but for XGBoost, both log transformed and original count features were kept for the model to choose from.

Lastly for NLP, the overall strategy was to combine the textual features (description, screen name and location) into one field, leaving out the url features. Then, use word level TF-IDF to capture word importances, but this leaves out positional information of the words, therefore character level TF-IDF was done as well. This then outputs a lot of features, approximately 30,000, after which dimensionality reduction was done to allow for more efficient training later on, using TruncatedSVD, which is able to work on the sparse arrays that the TF-IDF operations create. This had to be done carefully though, since TF-IDF can cause data leakage when used on the entire dataset, as frequencies are involved.

2.3. Model Development and Validation

A total of 12 models were trained, which was worked on iteratively in increasing order of complexity. The more basic models included logistic regression and random forest. Logistic regression used structured features and the log-transformed count features without the original ones. Next was random forest, which was trained on all structured features.

Next were gradient boosting methods. XGBoost is known to be robust to class imbalance and interpretability, trained on structured features with early stopping. LightGBM has faster training and efficient handling of wide datasets, while CatBoost is able to handle categorical variables natively, though we did not take advantage of that given that we already one-hot encoded the categorical lang in processing. A variant of XGBoost was also trained to incorporate NLP features to give textual signals, as outlined by the NLP strategy above.

A neural network (MLP) was then trained using Keras, with experiments on structured-only, text-only, and combined features. Since combining all features provided the best performance, that was the only MLP model to be further developed going forward with the additional techniques.

2.4. Additional Techniques

While logistic regression and random forest models used a simple 80-20 test split, the rest of the models used stratified cross-validation (CV), which improves robustness and avoids overfitting to the train data. The gradient boosting models used a 5-fold CV while NN models used a 3-fold cross validation for higher efficiency.

Gradient boosting models and NN models had hyperparameter optimization, initially using GridSearchCV provided by ScikitLearn but switched to Optuna which was much more efficient due to its ability to leverage bayesian optimization and efficient pruning strategies with sampling algorithms

Next semi-supervised learning was done through pseudo-labelling, which used the strongest model at the time (XGBoost). Samples with high predictions in the train set ($p<0.05$ and

p>0.95) were added to the training data, increasing its volume by about 20%. All three gradient boost models and the MLP model were retrained on this pseudo-labelled dataset.

With these four models trained on the pseudo-labelled data, I employed ensemble strategies to try to improve accuracy and performance, also increasing robustness by reducing overfitting. A weighted average ensemble combined the four models and retrained on the pseudo-labelled dataset with Optuna and 5 fold CV to find the best model weights. Then, instead of a simple weighted average, a meta ensemble was created, with XGBoost as the meta model since it previously performed the best. Some experimentation was done with other models as well and it was found that XGBoost was the best meta model.

3. Results and Discussion

3.1. Model Evaluations and Final Strategy

To evaluate these models, the studies, models and submission files were loaded back into the notebook. Unfortunately for logistic regression and random forest, those models did not have an Optuna study and therefore were difficult to evaluate. Though, not much effort was put into those models and they did not perform well initially. The notebook's evaluation section covered feature importances, hyperparameter importances, and comparisons between models to see which techniques helped out.

The NLP features actually worsened models, apart from MLP,. Pseudo-labelling was also able to consistently provide an approximate 1.7% improvement in CV ROC AUC score as well. The meta ensemble also performed the best on the CV ROC AUC score, while the weighted ensemble does not have a proper Optuna study that can be analysed. The final strategy is to select the best structured features only model (XGBoost) alongside the best model (XGBoost meta ensemble). I did not focus too much on the leaderboard accuracy as it is only 30% of the train data, and the top 50 positions have a difference of about 1%, which might be attributed to noise.

4. Conclusion and Reflection

This project demonstrated that fraud analysis models benefit both from domain understanding such as from the strong improvements from feature engineering, and the use of advanced techniques like pseudo-labelling or ensembling. That said, planning out my models and methodology would have helped significantly, as I had to combine all my notebooks into one close to the submission date, which was a rush to rerun everything. Detailed planning would also help me better keep track of all my models, and ensure that I am iterating in the right direction. All in all, I learnt a lot on how to approach bot detection and how to approach fraud problems with machine learning.